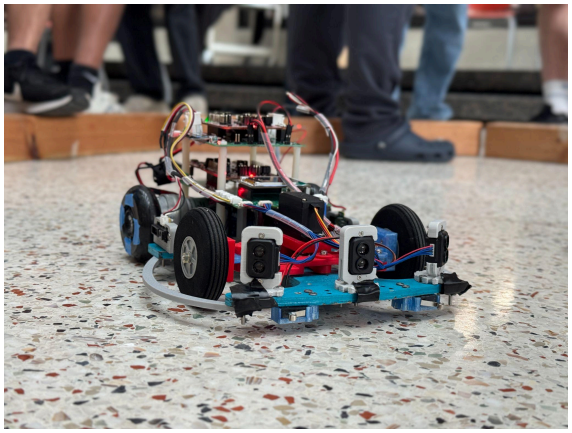
1. Show us a cool thing you've built/coded up :) I know you're already submitted a portfolio, but this one should be something that you're especially proud of and want to explain why. For reference a single image and a three sentence description explaining why it's cool / unusual is perfect. If it is a school project explain what class it was for

 This is George, an autonomous race robot I helped build for my Real-Time Embedded Systems Lab. I helped develop the RTOS and wrote the threads responsible for Wi-Fi communication and motor control, which taught me a lot about designing reliable real-time systems. George was a bit of an underdog, we placed in the bottom four teams out of 12 during testing, but because we focused on solid fundamentals and reliability rather than pushing speed and crashing, George finished **3rd on race day**, which is something I'm really proud of.

2. Describe your setup. What text editor / IDE / cosmic ray do you use? Why do you like it? How has it changed over time?

I use VS Code for most of my development because I'm comfortable with it and like having my code, terminal, Git, and debugging tools all in one place. If I'm doing a quick microcontroller interface or testing an idea, I'll often start with the Arduino IDE because it lets me get something running quickly without much setup. As the project becomes more serious, I usually move over to VS Code for better project organization, debugging, version control, and overall development workflow.

3. You get a PCB that resets randomly once every few hours. What are the steps you take to debug it? What are your go-to first couple of tests? When do you change strategies? Bonus points if you can tell a story about a real incident

My first step would be to check the power rails with a multimeter and make sure they are at the expected voltages. If everything looks normal, I would move to an oscilloscope and look for voltage dips, noise, or transients that could be causing a reset. I'd also check the reset and brownout signals and, if possible, monitor current draw to see if there is a pattern when the reset occurs.

At the same time, I would go back through the schematic and PCB layout to understand what could realistically cause the failure. If I couldn't find anything on the power side, I would start changing strategies and look at communication signals, firmware, and environmental conditions.

I ran into a similar issue while testing the Chessbot PCB. We were trying to get a TMAG magnetometer communicating over I2C. We first verified that the sensor was receiving the correct power, so we knew the problem wasn't simply that the device wasn't powered. We then connected a logic analyzer and looked at the SDA and SCL lines. We were seeing the communication, but the sensor wasn't sending an ACK when we expected it to.

Since the power looked correct, we went back to the schematic and PCB design instead of continuing to debug the firmware. We discovered that we had used an outdated TMAG footprint where the SDA and SCL pins were labeled inversely. Once we corrected the connection, the magnetometer started communicating correctly. It reinforced for me that when debugging hardware, I don't want to assume the problem is software just because I'm testing through software, I want to systematically work from the physical signals back up through the system.

4. What do you think firmware/embedded engineers do wrong? What are common words of wisdom/best practices you disagree with? Why?

I think firmware engineers can sometimes spend too much time trying to understand the entire system before writing or testing anything. I prefer to iterate quickly: get a small piece working, test it on the actual hardware, see what breaks, and learn more about the system as I go. In embedded systems, I've found that actually interacting with the hardware often teaches me things that I wouldn't have understood just by reading the documentation first. I still think it's important to understand the bigger picture, but I'd rather build my understanding alongside the system than wait until I feel like I understand everything before starting.

5. Have you ever hacked together a firmware "fix" you knew was kinda sketchy but it kept the project alive? What was it? Bonus points if it's one you're secretly proud of.

   While developing a real-time operating system, we were seeing an extremely large jitter spike from an ISR that only happened once. We were tracking jitter in CPU cycles using a histogram to determine whether our system was on track to pass the lab, and that one outlier was throwing everything off. So, in true engineering fashion, we added a check that if the maximum jitter exceeded a certain threshold, we recorded it as a different value in the histogram. It was definitely not the most “real-time” solution, but it kept our results on track and we passed the lab. I’m not sure I’m proud of the fix, but I am a little proud that it worked.